Project Report

Yongnan Zhong (yz3483), Ziwei Liu (zl2254), Peijie Li (pl675), Shuo Zhang (sz725)

GitHub repository link: https://github.com/cyan0895-lgtm/cs5782-final-project

Paper reproducing: Wu, F., Souza, A., Zhang, T., Fifty, C., Yu, T., & Weinberger, K. (2019). Simplifying Graph Convolutional Networks. Proceedings of the 36th International Conference on Machine Learning (ICML).

Introduction

This paper introduces Simple Graph Convolution (SGC), showing that GCNs can be simplified by removing nonlinearities and collapsing weight matrices into feature propagation followed by a linear classifier. SGC works as a fixed low-pass filter that smooths node features over neighbors, making the model faster, smaller, easier to interpret, and still comparable in accuracy to more complex GNN models. We chose this paper because we were interested in whether a simpler model could complete the same graph learning task effectively. Since SGC suggests that much of GCN's success comes from graph propagation rather than model complexity, we wanted to reproduce the results and better understand how smoothing features plus a linear classifier can achieve strong performance.

Chosen Result

We reproduce the node classification experiments on citation networks datasets (Cora, Citeseer and Pubmed) and Reddit dataset, focusing on the key finding that SGC can achieve performance comparable to GCN on node classification while significantly improving training efficiency by removing nonlinearities and simplifying propagation. We also run GCN and FastGCN as comparison baselines to evaluate the tradeoff between model efficiency and performance. Model performance was evaluated using test accuracy for citation networks and Micro-F1 for Reddit.

Beyond the node classification, we further explore downstream tasks including text classification and graph classification to evaluate the generalization of SGC across different tasks. For text classification, we conduct experiments on the R8 dataset and include GCN as a comparison baseline. For graph classification, we evaluate the performance of SGC, GCN, and GIN on the NCI1 dataset. These experiments allow us to analyze when simplified linear propagation is sufficient and when more expressive nonlinear graph models are still necessary.

Methodology

We reproduced the node classification setting from Simplifying Graph Convolutional Networks. The main idea is to simplify a standard GCN by removing intermediate nonlinearities and collapsing multiple weight matrices into one linear classifier. A standard GCN layer can be written as: $H^k = ReLU(SH^{(k-1)}\Theta^{(k)})$, where S is the normalized adjacency matrix with self-loops, $H^{(k-1)}$ is the node representation from the previous layer, and $\Theta^{(k)}$ is the trainable weight matrix. In SGC, the nonlinear activation functions are removed, and the repeated graph propagation is precomputed as: $\bar{X} = S^K X$. Then, the final prediction is made by a simple linear softmax classifier: $\hat{Y} = softmax(\bar{X}\Theta)$. The overall model architecture is shown in Figure A1. The implementation was completed with Python, PyTorch, and PyTorch Geometric.

Results & Analysis

Citation Network & Reddit

Table A1 shows that SGC achieves competitive performance across all three citation datasets while training substantially faster than GCN and FastGCN. SGC achieves the best accuracy on Citeseer and performs comparably to other models on Cora and Pubmed despite using a much simpler linear architecture. Similar to the paper (Figure A2), SGC achieves the best trade-off between efficiency and performance because neighborhood propagation is precomputed before training, reducing the model to a simple linear classifier.

For the Reddit experiments, our reimplementation successfully reproduces the main findings of the paper. Figure A3 shows that SGC converges significantly faster than GCN and FastGCN while maintaining competitive MicroF1. During reproduction, we also encounter GPU memory limitations due to the large scale of the Reddit graph during full graph propagation. As a result, the standard full batch GCN implementation runs out of memory, so we mainly compare SGC with FastGCN on Reddit experiments.

Downstream Tasks

To evaluate whether SGC generalizes beyond node classification, we test it on the R8 text classification dataset. Consistent with the original paper, SGC outperforms GCN while training substantially faster. Following the original setup, we construct a word-document graph and apply SGC with K=2. Our reproduced result (92.42%) is lower than the reported 97.2% (Table A2), mainly because GPU memory limitations required TruncatedSVD reduction from 7,688 to 800 dimensions, introducing information loss. Nevertheless, SGC still significantly outperforms GCN, supporting the paper's conclusion.

Besides text classification, we also evaluate SGC on graph classification using the NCI1 dataset. Figure A4 shows that although SGC remains computationally efficient, it underperforms more expressive graph models such as GCN and GIN. These results suggest that SGC works well for homophilous and smoothing-dominant tasks, but becomes limited for graph tasks that require stronger structural representation learning and nonlinear feature extraction.

Future Work

We further extend SGC with an Adaptive K mechanism, where the model learns weighted combinations of multiple propagation depths instead of using one fixed K. Despite introducing adaptive receptive fields, the model maintains similar performance comparable to fixed-K SGC on Reddit. Figure A5 and Figure A6 show that the Adaptive model assigns the highest weights to shallow propagation depths (K=1 and K=2), while deeper propagation receives smaller weights, suggesting that local neighborhood information is most important and overly deep propagation may cause oversmoothing.

Reflections

Through this project, we learn that SGC performs very well and greatly improves training efficiency by removing nonlinearities and simplifying propagation on homophilous node classification and text classification tasks, where neighborhood feature smoothing is the dominant factor. However, our graph classification experiments show that SGC underperforms more expressive models such as GCN and GIN, which suggests that nonlinearities may be unnecessary for some smoothing-dominant tasks, but they remain important for graph tasks that require stronger structural representation learning and more complex feature interactions. Overall, the project helps us better understand the trade-off between model simplicity, efficiency, and expressive power in graph neural networks. Based on our findings, a potential future direction is to design adaptive propagation mechanisms that preserve the efficiency of SGC while improving its flexibility across different graph tasks.

References

Wu, F., Souza, A., Zhang, T., Fifty, C., Yu, T., & Weinberger, K. (2019). Simplifying Graph Convolutional Networks. Proceedings of the 36th International Conference on Machine Learning (ICML).

Kipf, T. N., & Welling, M. (2017). Semi-Supervised Classification with Graph Convolutional Networks. International Conference on Learning Representations (ICLR).

Defferrard, M., Bresson, X., & Vandergheynst, P. (2016). Convolutional Neural Networks on Graphs with Fast Localized Spectral Filtering. Advances in Neural Information Processing Systems (NeurIPS).

Hamilton, W. L., Ying, Z., & Leskovec, J. (2017). Inductive Representation Learning on Large Graphs. Advances in Neural Information Processing Systems (NeurIPS).

Chen, J., Ma, T., & Xiao, C. (2018). FastGCN: Fast Learning with Graph Convolutional Networks via Importance Sampling. International Conference on Learning Representations (ICLR).

Perozzi, B., Al-Rfou, R., & Skiena, S. (2014). DeepWalk: Online Learning of Social Representations. Proceedings of the 20th ACM SIGKDD Conference on Knowledge Discovery and Data Mining.

Grover, A., & Leskovec, J. (2016). node2vec: Scalable Feature Learning for Networks. Proceedings of the 22nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining.

von Luxburg, U. (2007). A Tutorial on Spectral Clustering. Statistics and Computing.

Appendix

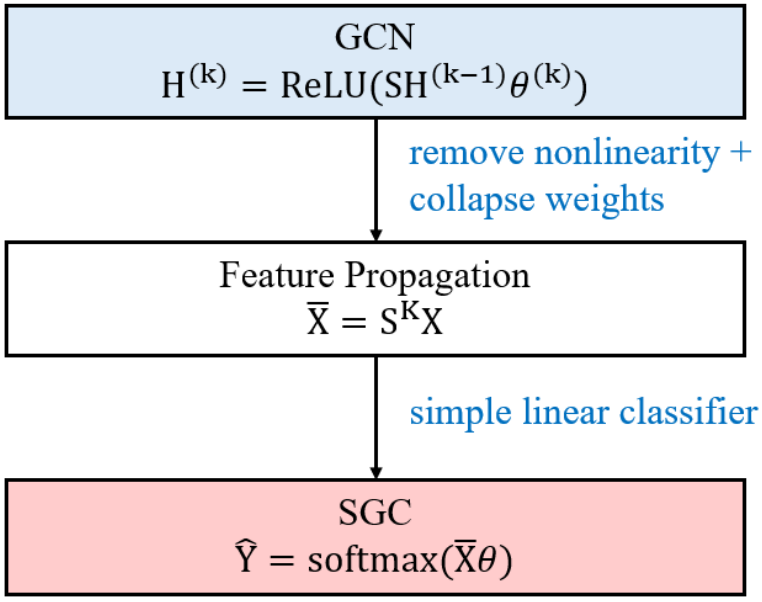

Figure A1: Methodology overview of simplifying GCN into SGC

Table A1: Test accuracy (%) averaged over 10 runs on citation and reddit networks

Model	**Cora**		**Citeseer**		**Pubmed**		**Reddit**	
	Acc(%)	Time	Acc(%)	Time	Acc(%)	Time	Micro-F1(%)	Time
SGC(ours)	82.0 ± 0.0	0.2s	71.8 ± 0.0	0.2s	78.9 ± 0.0	0.2s	94.6	5.7s
GCN	81.4 ± 0.9	0.8s	68.9 ± 1.2	0.9s	79.1 ± 0.5	1.2s	OOM	OOM
FastGCN	82.1 ± 1.2	1.4s	68.3 ± 1.0	2.3s	76.0 ± 0.8	2.8s	90.4	1581.9s

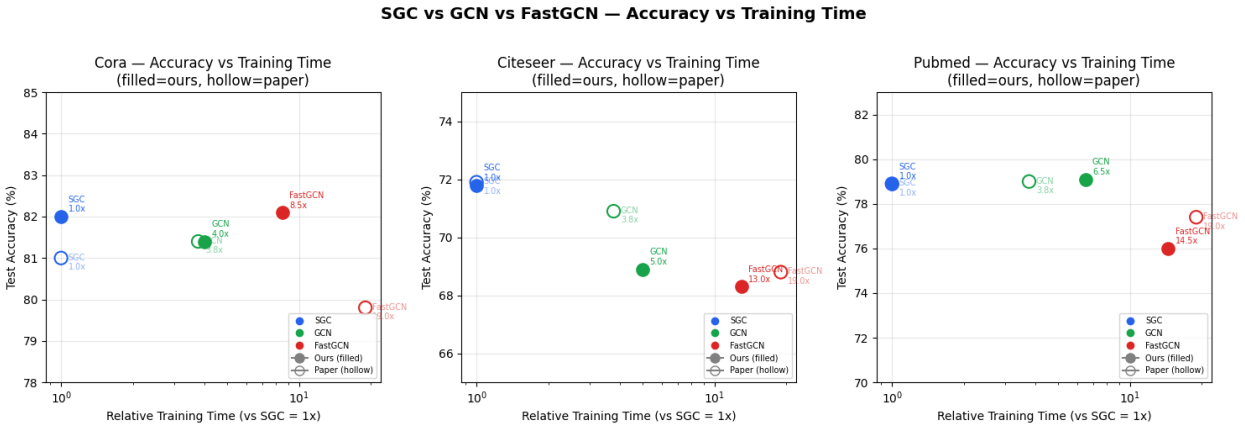

Figure A2: Accuracy and Training Time of SGC, GCN and FastGCN

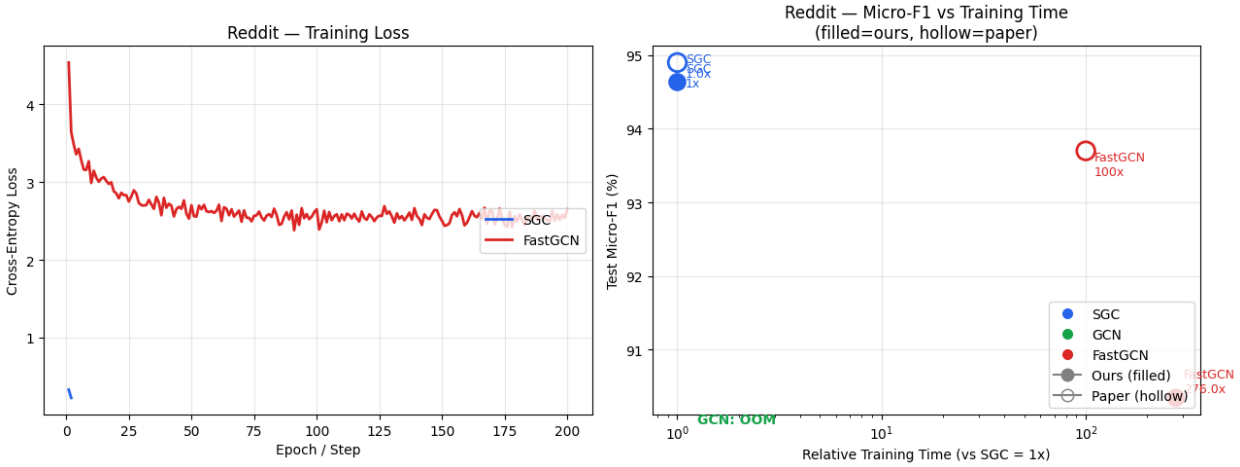

Figure A3 Training and Performance Comparison on Reddit

Table A2: Test Accuracy (%) on R8 text classification datasets

Model	**R8**		**R8 (paper)**	
	Acc(%)	Time	Acc(%)	Time
SGC(ours)	92.4	18s	97.2	2s
GCN	77.3	64s	97.0	130s

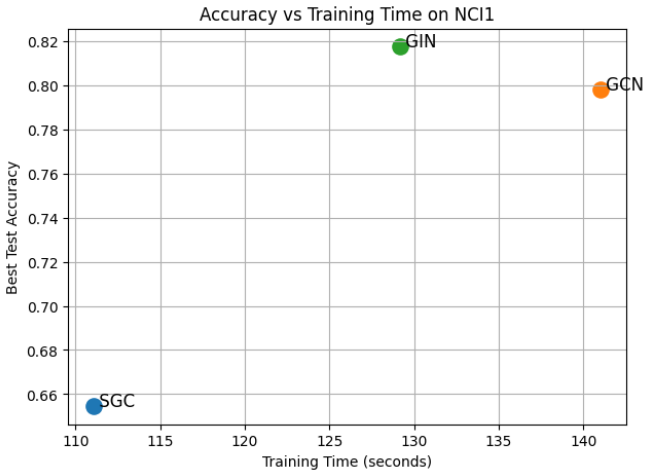

Figure A4 Accuracy vs Training Time om NCI1

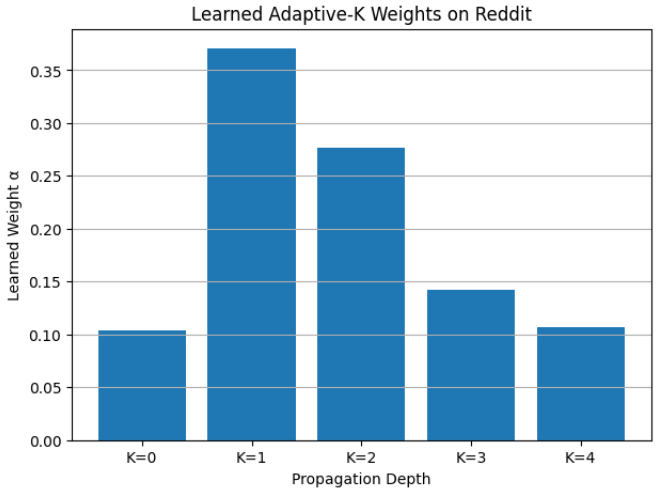

Figure A5 Learned Adaptive-K Weights on Reddit

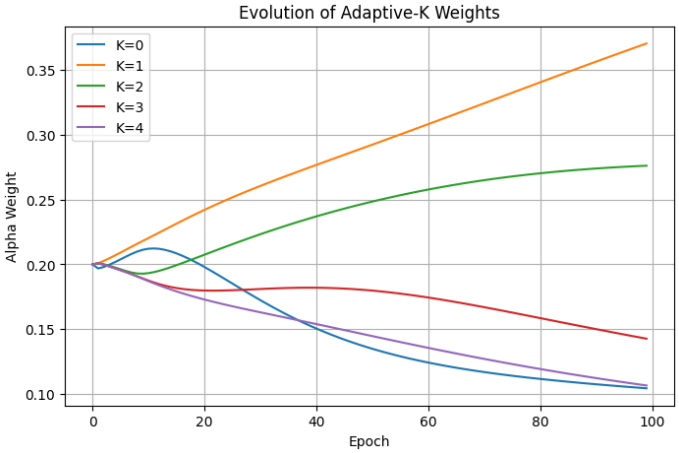

Figure A6 Evolution of adaptive propagation weights during training